

MC-core request / logic flow — AXI to DFI

One channel, DDR5. Module-level sub-blocks by name; only the load-bearing seam signals shown. Per-rank lane = ingress..rank-arb, replicated xN_RANKS; L3 + back-end + scoreboard + maint shared. Handshake shorthand: v/r=valid/ready, f/e=full/empty, oh=one-hot grant, bc=broadcast (no handshake), ae=async CDC FIFO.

Spine: CIF =ae=> bp_FIFO -> WLR -> LOOKAHEAD -> BANK_DEC -> [BANK_CELL x32 + FSM + ELIG] -> CMD_ARB -> BGB/BG_ARB -> RANK_ARB -> L3 -> CMD_PHASER -> DFI_ENC -> PHY

1 Command / request path (per rank lane, xN_RANKS)

module	role	interface to next (handshake + meta/data)
<i>ingress</i>		
CIF (host)	segment <=16-beat pkts, hash addr->daddr, alloc rob_index + sram slot	OUT: reqrob_index, daddr, wd_slot dbuf_addr, op v/r over CDC
R_REQ_AFIFO / W_REQ_AFIFO	async in, the only true CDC	OUT: same rec, synced to mc_clk; v/r (ae)
RD_BP_FIFO / WR_BP_FIFO	per-rank back-pressure, split by dir (depth 16)	OUT: rob_index, phy_adr, wr_data_prt; flags f/partial_f -> TURN_CTRL, e -> arb skip
TURN_CTRL + WLR_SEL	weighted R/W batch, one merge point	OUT: one combined stream + batch_dir; ctrl rd/wr_select (mux)
LOOKAHEAD	4-entry bypass window (e0..e3)	IN: bank_rdy[N_BANKS-1:0] (bc). OUT: rob_index, phy_adr, op head; shift on sel
BANK_DEC	1-of-N_BANKS decode rank,bank	OUT: per-bank push strobe; consumes bank_rdy
<i>per-bank cell (xN_BANKS per lane)</i>		
BANK_REQ_REG[r,b]	1-deep payload hold	OUT (at CAS-commit, via grant_oh): op, rob_index, col, sram_addr -> CMD_PHASER
BANK_STATUS_REG[r,b]	state[4], open_row, valid, pre_pending	OUT: fields -> ROW_COMPARATOR + ELIG (bc read)
ROW_COMPARATOR[r,b]	pkt.row vs open_row	OUT: is_hit, need_pre/act/cas -> ELIG
BANK_ELIG[r,b]	need_* & can_* fold	IN: can_* (scoreboard bc), stall_acts. OUT: bank_go, bank_cmd(cas/act/pre), is_hit -> CMD_ARB
BANK_FSM[r,b]	6-state cmd-progress FSM	IN: grant_* (issue-edge), can_cas/can_act, gate_rfc/rfm. OUT: state, cas_complete, bank_busy, row_valid, act/cas/pre_set (scoreboard writeback)
<i>arb tree (selection only; legality pre-folded in can_*)</i>		
CMD_ARB[r,b]	tier-1: pick cmd for bank	IN: CAS_W/ACT_W/PRE_W. OUT: go, cmd, is_hit -> BGB_ARB
BGB_ARB / BG_ARB	bank-in-BG (4->1), BG-in-rank (8->1)	OUT: per-rank per-class candidate; BG_ARB weight = diff-from-last_cas_bg
RANK_ARB (act/cas/pre)	rank_act_abr, rank_cas_abr, pick-2-pre	OUT: act_win, cas_win, pre_win[0..1] = rdy, bg, bank -> SCHEDULER
<i>schedule + issue (shared)</i>		
SCHEDULER + L3	class selectors (cas_L3 +dqFree/tRTRS, act_L3, pre_L3), pick <=2	IN: winners + maint flags + turnaround. OUT: 2 slots vld, len, CMD_PACKET; grant_*_oh (pop-on-CAS), fsm_commit, cmd_issued
CMD_PHASER	2-slot cmd_wire (not a fifo)	IN: 2 slots + gear. OUT: cmd_0 -> ENCO, cmd1 -> ENC1; rdy0/rdy1 back-pressure -> scheduler
DFI_CMD_ENCODER x2	OPCODE_ROM + FIELD_MUX + CS_GEN	IN: record + gear_cfg (always_on / b1_enable). OUT: ENCO->dfi_p0/p1, ENC1->dfi_p2/p3; 1:1 temporal mux (2bit_wrap_cnt)
DFI CMD PORTS	per-phase drive	OUT: dfi_address/cs_n/cke/odt/parity_pN -> PHY -> DRAM

2 Write data path (shared)

module	role	interface to next (handshake + meta/data)
AXI W channel	beats in	OUT: wdata, wstrb, wlast v/r
WR_ACCUM	beats -> 512b line, WSTRB->DM	IN: narrow flag (from WR_ROB). OUT: 512b line + DM -> WD_SRAM (one wide write on wlast); alloc line on first beat
WD_SRAM	512b line store (1 pkt/line)	IN: write @ wd_slot. OUT: 512b read @ wd_slot (on WL fire)
WL_LAUNCH	write-latency delay line	IN (at CAS-commit): wd_slot, rob_index, tphy_wrlat. OUT: dfi_wrddata_en @ cmd+tphy_wrlat; fires WR_SPLIT
WR_SPLIT	512b -> 2*gear beats/mc_clk	IN: 512b, gear. OUT: dfi_wrddata_pN @ ent+tphy_wrddata; wr_done (self-timed, last beat)

3 Read data path (shared)

module	role	interface to next (handshake + meta/data)
DFI rddata in	PHY drives	IN: dfi_rddata_pN + dfi_rddata_valid
RD_CAP	RL delay line	IN (at RD-issue): dbuf_addr, rob_index, tphy_rdlat. OUT: capture window enable -> RD_ACCUM
RD_ACCUM	2*gear beats -> 512b line	OUT: 512b line -> RD_SRAM @ dbuf_addr (one wide write)
RD_SRAM	512b line store	OUT: 512b read @ dbuf_addr -> RD_SPLIT
RD_SPLIT	512b -> AXI R beats	OUT: rdata, rlast v/r -> AXI R channel

4 Completion / tracking (shared)

module	role	interface to next (handshake + meta/data)
COMPLETION	merge rd_done / wr_done	IN: rd_donerob_index,pkt (last beat in RD_SRAM), wr_donerob_index,pkt (self-timed). OUT: rob_index, pkt_num, status -> resp AFIFO
R_RESP_AFIFO / W_RESP_AFIFO	async out to CIF	OUT: tag-only completion v/r (ae) -> CIF
WR_ROB	write tracker vld, N, gen_idx, done_cnt	gen_idx -> addr offset (w_phy_adr + gen_idx*64B); retire when done_cnt==N -> COMPLETION

5 Scoreboard (feeds ELIG + arb; not in the data stream)

module	role	interface to next (handshake + meta/data)
GC	24b free-run, +=gear each mc_clk	OUT: GC -> all UPD adders + comparators (bc)
TIMING_CONST	JEDEC addends (20 x 10b, one copy)	OUT: t -> every *_UPD adder
UPD	next* = GC + const, per scope (bank/BG/rank/ch)	IN: cmd_issued (WE + router). OUT: 13b deadline -> *_TREG
_TREG + CMP	deadline store + GE compare	OUT: can_act/can_cas/can_pre (1b) -> ELIG (bc); can_*_bgs/rank -> arb
refi_cnt + debt	14b down-counter + 3b debt	OUT: ref_rdy= debt, ref_force=&debt -> MAINT/scheduler
RAA_cnt	per-bank ACT counter	OUT: rfm_rdy (>=RAAIMT), rfm_force (>=RAAMMT) -> MAINT

6 Maintenance (peer source; injects, never CAS)

module	role	interface to next (handshake + meta/data)
MAINT_ENG	INIT / REFRESH / RFM / ZQ / MRW / MRR sub-engines	IN: cas_complete[banks] (drain done), debt/RAA (scoreboard), MR4 poll. OUT: stall_acts -> banks (quiesce), gate_rfc/rfm -> scoreboard mask, maint cmd -> CMD_PHASER (override), init_done -> DFI_MUX

Gaps (revisit): LOOKAHEAD-to-BANK_DEC decision in full logic space (bank_cmp / prio-enc detail); per-rank vs shared boundary of a couple scoreboard slots. Flagged, not blocking. **Config (APB/MR/PHY/FREQ_FSM)** writes CONFIG_REGS / MR_SHADOW / tphy_* out-of-band (boot / DVFS-quiesced / MR-poll) — not on the request path; see config_regs.pdf.